PORTFOLIO · INFRASTRUCTURE / DEVOPS

kctl Kubernetes 배포 자동화 플랫폼

- 명령 한 줄 → 빌드 · 이미지 push · 매니페스트 생성 · nginx 라우팅 · 롤아웃 자동 처리
- 개발자 CLI(`kctl`) + 노드 에이전트(`kctl-agent`) 두 바이너리 구성

기간	역할	규모	운영
2025.05 - 현재	1인 · 설계 / 구현 / 운영	Go 약 5,400줄 · 바이너리 2개	실서버 k3s · 서비스 3개

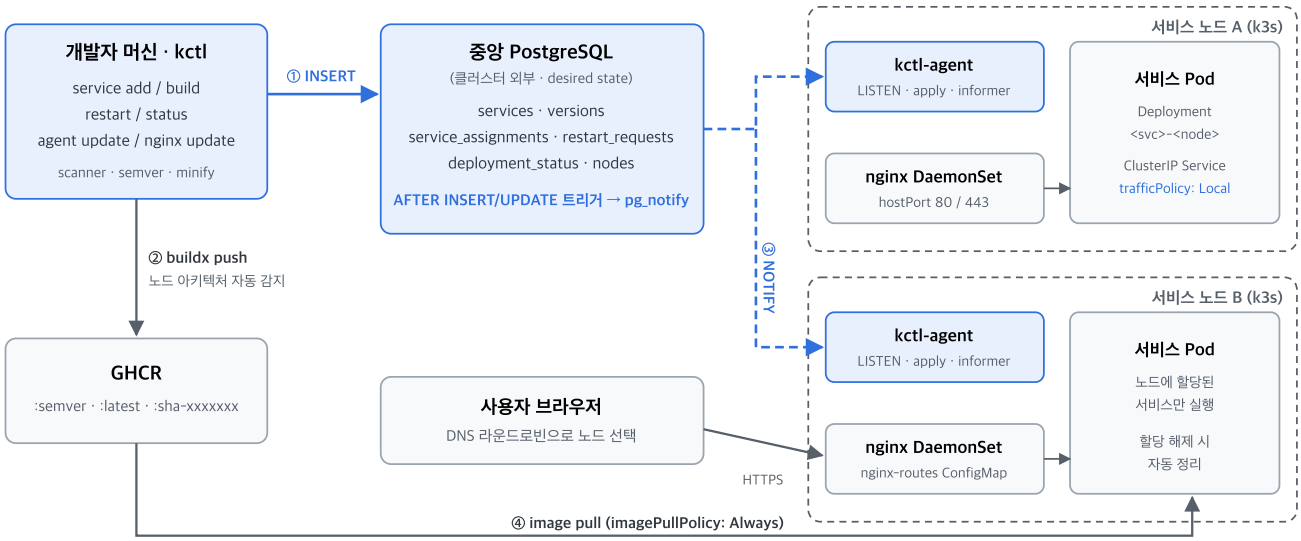
Go 1.22 · Cobra / Viper · PostgreSQL LISTEN/NOTIFY · k3s · client-go Informer · Docker buildx · GHCR · nginx DaemonSet · SOPS + age · systemd

AI 사용 범위 · 프론트엔드

문제 정의

- 서비스마다 Dockerfile · Deployment · Service · nginx 설정 · 인증서 · 이미지 태그 수작업 → 실수 한 번이 장애로 직결
- 노드 간 네트워크 느림 → 마스터 SPOF 회피, 노드 간 트래픽(hop) 제거 필요
- Kubernetes = **자동 분산 X, 배포·관리 자동화 도구**로 한정
- 서비스 성격별 노드 직접 배치 + DNS 라운드로빈 트래픽 분산

아키텍처



kctl = DB에 원하는 상태만 기록 · kctl-agent = NOTIFY 수신 후 자체 반영(pull 방식) · 노드 관리 포트 0개

주요 기능

명령	동작
`kctl service add`	Dockerfile · `k8s/*.yaml` · git remote 스캔 → 포트·리소스·볼륨·nginx 설정 자동 감지 → DB 등록 + 첫 빌드 · GHCR push · 배포 일괄 처리
`kctl build`	semver bump → `:semver` · `:latest` · `:sha-7` 3개 태그 push · 진입 시 설정 재스캔 → DB 반영 · JS/CSS는 빌드 컨텍스트 복사본에서만 minify
`kctl status`	원하는 상태 + 실제 상태(READY / PHASE / LAST_SEEN) 동시 표시 · 에이전트 informer로 수 초 내 반영
`kctl restart`	`restart_requests` INSERT → NOTIFY → 대상 노드 에이전트 `rollout restart` · SSH 미사용
`kctl agent update`	노드 플랫폼별 크로스 컴파일 → scp → `install` → `systemctl restart` · 옵션으로 스키마 마이그레이션 선적용

`kctl service assign`	노드 할당 변경 · 할당 해제 노드는 라벨 기준 Deployment 자동 정리

설계 결정과 근거

PG LISTEN/NOTIFY 기반 pull 배포

- 메시지 브로커 · 노드 HTTP 서버 · 열린 포트 불필요
- 트리거 `pg_notify` → 에이전트 멱등 `kubectl apply`
- NOTIFY 유실 대비 5분 주기 전체 reconcile

노드별 nginx · 트래픽 노드 내 격리

- nginx DaemonSet `hostPort 80/443` 직접 listen → NAT 없음, 클라이언트 IP 보존
- Service `internalTrafficPolicy: Local` → 노드 간 이동 차단
- 엔드포인트 없으면 502 → DNS 라운드로빈 다음 노드로 (의도된 격리)

DB는 클러스터 밖

- 클러스터 장애 시에도 원하는 상태·이력 보존 → 재구축 기준점
- 노드 부팅 시 hostname으로 자체 등록
- 기존 행이면 운영자가 수정한 host/role/env 보존

바이너리 분리 · SOPS 시크릿

- 개발 머신 `kctl` 만, 노드 env 기반 `kctl-agent` 만 배치
- SOPS + age 암호화 → 평문 비밀번호 제거
- sudo env 제거 상황 대비 age 키 파일 자동 탐색

기술적 도전과 해결

01 한 도메인을 여러 서비스가 공유하는 nginx 라우팅 자동화

문제 라디오 UI(:8000) + 스트림 서버(:8001)가 `radio.doyoulike.xyz` 공유 → 서비스별 nginx 파일 시 `server_name` 충돌

해결

- 서비스별 `k8s/nginx.conf` DB 저장
- generator: 정규식 + 중괄호 카운터로 `server` / `location` 블록 추출 → 같은 `server_name` 끼리 병합
- `${SVC_DNS}` → location 소유 서비스 FQDN 치환
- ConfigMap 해시 annotation 비교 → 변경 시에만 nginx 재시작

결과

- 서비스 설정 파일 수정 + `kctl build` 1회로 라우팅 반영
- 전역 HTTP→HTTPS 리다이렉트용 `listen 80` 변형 자동 제거

02 `kctl status` 반영 지연: 5분 → 수 초

문제 상태 보고가 5분 주기 reconcile에만 의존 → 배포 직후 상태 확인 불가

해결

- client-go SharedInformer로 Deployment · Pod watch
- phase/readiness 실제 변경 이벤트만 통과 → 이벤트 폭주 차단
- 2초 debounce 후 `deployment_status` UPSERT + heartbeat

결과

- 반영 시간 수 초로 단축
- informer 실패 시 5분 reconcile 자동 fallback → 무중단
- 트레이드오프: 바이너리 +16MB

03 스캔 무음 실패로 어긋난 배포

문제

- YAML 문법 오류 시 스캔 skip → 새 이미지 + 낡은 DB 설정 → 포트·리소스 불일치 배포
- `deployment.bak.yaml` 포트 혼입

해결

- 파일 부재는 허용, 읽기·파싱 실패 시 빌드 중단
- 첫 Deployment 문서만 채택
- multi-stage Dockerfile은 마지막 인식된 FROM 기준 언어 판정
- 회귀 테스트 추가

04 운영 중 배포 사고 → 플랫폼 기능으로 흡수

OOM	liquidsoap · ffmpeg · supervisord 단일 컨테이너가 하드코딩 128Mi 제한으로 `OOMKilled` (137) → 서비스별 resources 지원 (`yaml.Node` 재직렬화 → DB → 템플릿 삽입)
캐시	같은 태그 재push 미반영 → `imagePullPolicy: Always` (manifest HEAD 1회 비용)
아키텍처	ARM 노드 `no matching manifest` → 할당 노드 `nodes.platform` 합집합으로 buildx 플랫폼 자동 결정
디스크	에이전트 이미지 정리 루프(부팅 10분 후 + 24시간마다 `crictl rmi --prune`) → containerd 4.8GB → 1.4GB

성과

명령 1줄 서비스 등록부터 공개 배포까지	**~수 초** 상태 반영 (기존 5분)	**포트 0개** 노드 관리용으로 열린 포트	**4.8→1.4GB** 이미지 자동 정리 효과

- 실서버 운영 서비스 3개: **AudioStream** · **stream** · **Scriptorium**
- Scriptorium 60회 이상 릴리스 (이 파이프라인 사용)
- 노드 부트스트랩(k3s join · kubeconfig fetch · SOPS bootstrap · systemd 등록) + 초기화 스크립트로 노드 재구축 자동화